

LangChain

Chat Models Basics

1. model.invoke command sends the prompt and gets a response back.

```
# Invoke the model with messages
result = model.invoke(messages)
print(f"Answer from AI: {result.content}")
```

2. System-Human-AI-Human-AI.... Conversations

```
# SystemMessage:
# Message for priming AI behavior, usually passed in as the first of a sequence
# HumanMessage:
# Message from a human to the AI model.
messages = [
    SystemMessage(content="Solve the following math problems"),
    HumanMessage(content="What is 81 divided by 9?"),
]
```

3. By creating and updating the list with the chat prompts and responses, we can pass the entire list to the Chatbot so it has full context of the previous conversation.

```
chat_history = [] # Use a list to store messages

# Set an initial system message (optional)
system_message = SystemMessage(content="You are a helpful AI assistant.")
chat_history.append(system_message) # Add system message to chat history

# Chat loop
while True:
    query = input("You: ")
```

```

if query.lower() == "exit":
    break
chat_history.append(HumanMessage(content=query)) # Add user message

# Get AI response using history
result = model.invoke(chat_history)
response = result.content
chat_history.append(AIMessage(content=response)) # Add AI message

print(f"AI: {response}")

print("----- Message History -----")
print(chat_history)

```

4. We can upload our chat to the cloud so that we can access it from anywhere anytime using Firebase Database. Login into google using "gcloud auth application-default login" from the command prompt.

[5_chat_model_save_message_history_firebase.py](#)

Prompt Templates

1. Making a Template for prompts in the form of tuples.

```

# PART 3: Prompt with System and Human Messages (Using Tuples)
messages = [
    ("system", "You are a comedian who tells jokes about {topic}."),
    ("human", "Tell me {joke_count} jokes."),
]
prompt_template = ChatPromptTemplate.from_messages(messages)

```

2. The text inside { } is the text to be replaced, and we provide a dictionary with { key(value to be replaced): value(value to be replaced with) } for each filler.

```
prompt = prompt_template.invoke({"topic": "lawyers", "joke_count": 3})
print("\n----- Prompt with System and Human Messages (Tuple) -----\\n")
print(prompt)
```

3. System-Human-AI conversation can be structured in the form of a message list, or it can all just be one long text. I personally think that the structured message list is a better option.

```
# PART 3: Prompt with System and Human Messages (Using Tuples)
print("\n----- Prompt with System and Human Messages (Tuple) -----\\n")
messages = [
    ("system", "You are a comedian who tells jokes about {topic}."),
    ("human", "Tell me {joke_count} jokes."),
]
prompt_template = ChatPromptTemplate.from_messages(messages)
prompt = prompt_template.invoke({"topic": "lawyers", "joke_count": 2})
result = model.invoke(prompt)
print(result.content)
```

```
# PART 3: Prompt with Multiple Placeholders
print("\n----- Prompt with Multiple Placeholders -----\\n")
template_multiple = """You are a comedian who tells jokes about {topic}.
# Human: Tell me {joke_count} jokes.
# Assistant: """
prompt_template = ChatPromptTemplate.from_template(template_multiple)
prompt = prompt_template.invoke({"topic": "lawyers", "joke_count": 2})
result = model.invoke(prompt)
print(result.content)
```

Chains

1. We simply pass operations/tasks and their sequence to LangChain, such as:

```
chain = prompt_template | model | StrOutputParser()
```

2. This tells Langchain to take the PROMPT, give it to the MODEL then take the result and parse it to get just the CONTENT.
3. We can also create RunnableLambda and RunnableSequence to do the same thing as above:

```
# Create individual runnables (steps in the chain)
format_prompt = RunnableLambda(lambda x: prompt_template.format_prompt(**x))
invoke_model = RunnableLambda(lambda x: model.invoke(x.to_messages()))
parse_output = RunnableLambda(lambda x: x.content)

# Create the RunnableSequence (equivalent to the LCEL chain)
chain = RunnableSequence(first=format_prompt, middle=[invoke_model], last=parse_output)
```

4. We can make custom RunnableLambda functions and add those to the sequence to get the output and get it in the form that we desire. The key is Creativity and knowing Lambda functions.

```
# Define additional processing steps using RunnableLambda
uppercase_output = RunnableLambda(lambda x: x.upper())
count_words = RunnableLambda(lambda x: f"Word count: {len(x.split())}\n{x}")

# Create the combined chain using LangChain Expression Language (LCEL)
chain = prompt_template | model | StrOutputParser() | uppercase_output | count_words
```

Parallel Chains

5. We can build chains that run other chains in parallel.

```
# Create the combined chain using LangChain Expression Language (LCEL)
chain = (
```

```

prompt_template
| model
| StrOutputParser()
| RunnableParallel(branches={"pros": pros_branch_chain, "cons": cons_branch_chain})
| RunnableLambda(lambda x: combine_pros_cons(x["branches"]["pros"], x["branches"]["cons"]))
)

```

6. chain is the main chain. After it gets the content out of the result. It passes this content to the pros_branch_chain and the cons_branch_chain, that run in parallel too.

```

# Simplify branches with LCEL
pros_branch_chain = (
    RunnableLambda(lambda x: analyze_pros(x)) | model | StrOutputParser()
)

cons_branch_chain = (
    RunnableLambda(lambda x: analyze_cons(x)) | model | StrOutputParser()
)

```

7. As seen above, the lambda function is used to construct the prompt using a function.

```

# Define pros analysis step
def analyze_pros(features):
    pros_template = ChatPromptTemplate.from_messages(
        [
            ("system", "You are an expert product reviewer."),
            (
                "human",
                "Given these features: {features}, list the pros of these features.",
            ),
        ]
    )
    return pros_template.format_prompt(features=features)

```

8. We could also do this using a prompt template, but then we would also need to pass con and pro to the template as input to let it know when to tell pros and cons of the device.

Branching Chains

9. We can run different branches/chains based on requirements being met. It is like an if-else condition, but for chains. You could call it conditional chain running.
10. This is the hierarchy of the branched chains

```
### CALLING THE MAIN CHAIN using .invoke
review = "The product is terrible. It broke after just one use and the quality is ver
result = chain.invoke({"feedback": review})

### MAIN CHAIN
chain = classification_chain | branches

### SUB CHAIN
classification_chain = classification_template | model | StrOutputParser()

#CLASSIFICATION PROMPT TEMP
classification_template = ChatPromptTemplate.from_messages(
    [
        ("system", "You are a helpful assistant."),
        ("human",
            "Classify the sentiment of this feedback as positive, negative, neutral, or es
    ]
) #Output of this is Positive, Negative, Neutra or Escalate

### BRANCH CHAINS (4 POSSIBLE CHAIN OPTIONS)
# Define the runnable branches for handling feedback
branches = RunnableBranch(
    (
        lambda x: "positive" in x,
```

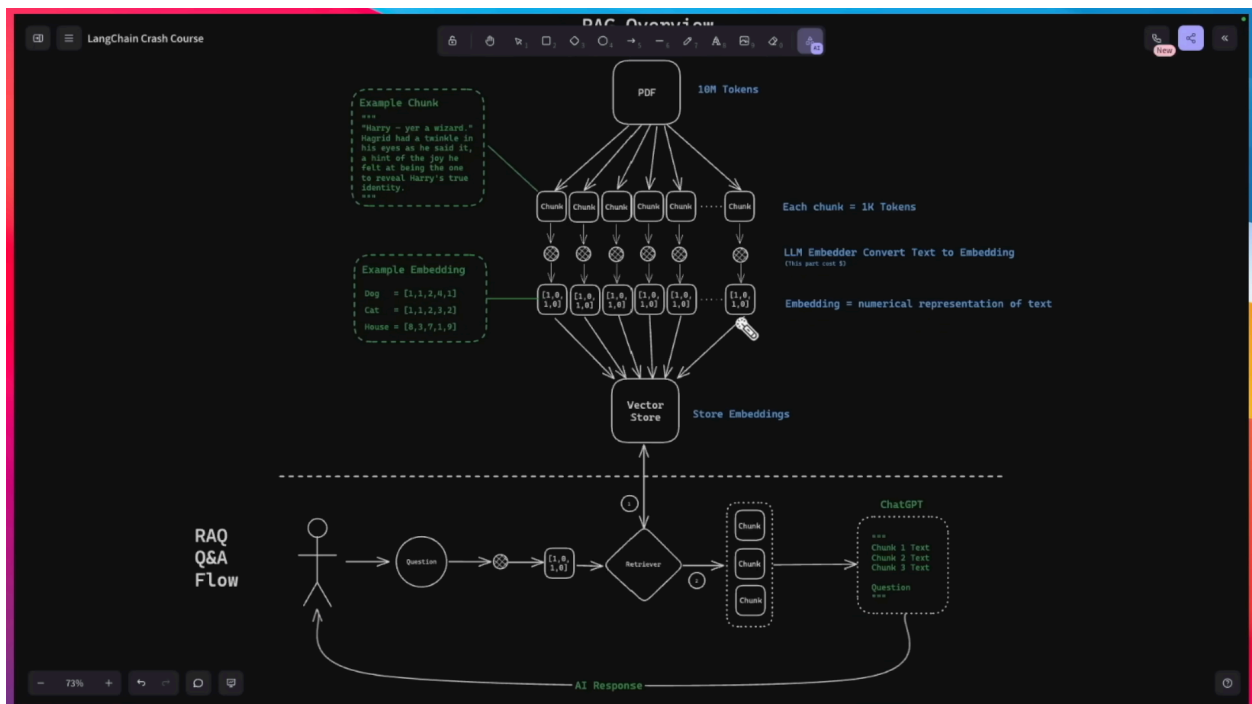
```

    positive_feedback_template | model | StrOutputParser() # Positive feedback
),
(
    lambda x: "negative" in x,
    negative_feedback_template | model | StrOutputParser() # Negative feedback
),
(
    lambda x: "neutral" in x,
    neutral_feedback_template | model | StrOutputParser() # Neutral feedback
),
escalate_feedback_template | model | StrOutputParser() # Default feedback chunk
)

```

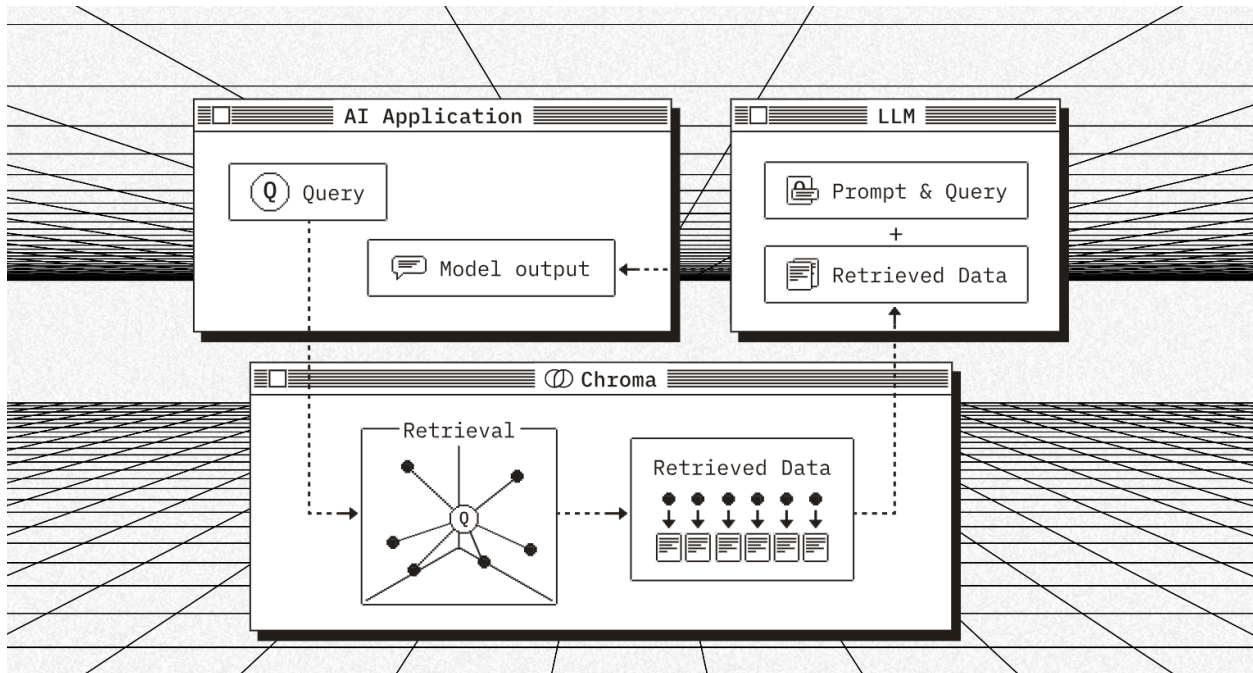
RAG

1. This is an overview of RAG



2. This is how Chroma works. It retrieves relevant data from a vector base (a bunch of embeddings) by checking their similarity with our query. The

similarity threshold can be set.



3. These files show the entire process and the file management processes, too.

- 1a shows how vector embeddings are produced from chunks of code

[1a_rag_basics.py](#)

- 1b shows the retrieval of similar data chunks after checking similarity of embeddings.

[1b_rag_basics.py](#)

Meta Data

4. We add source identification data (name of source, data, author, etc) to the source so that when it retrieves the particular chunk from the data source, it can give the source of the data chunk.

```
# Read the text content from each file and store it with metadata
documents = []
for book_file in book_files:
    file_path = os.path.join(books_dir, book_file)
    loader = TextLoader(file_path)
    book_docs = loader.load()
    for doc in book_docs:
        # Add metadata to each document indicating its source
        doc.metadata = {"source": book_file}
        documents.append(doc)

# book_file is the name of the book ending with .txt
```

Text Splits

5. There are several ways that text can be broken down into chunks.

6. Character-Based Splitting

```
# 1. Character-based Splitting
# Splits text into chunks based on a specified number of characters.
# Useful for consistent chunk sizes regardless of content structure.
print("\n--- Using Character-based Splitting ---")
char_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
char_docs = char_splitter.split_documents(documents)
create_vector_store(char_docs, "chroma_db_char")
```

7. Sentence-Based Splitting

```
# 2. Sentence-based Splitting
# Splits text into chunks based on sentences, ensuring chunks end at sentence k
# Ideal for maintaining semantic coherence within chunks.
print("\n--- Using Sentence-based Splitting ---")
sent_splitter = SentenceTransformersTokenTextSplitter(chunk_size=1000)
```

```
sent_docs = sent_splitter.split_documents(documents)
create_vector_store(sent_docs, "chroma_db_sent")
```

8. Token-Based Splitting

```
# 3. Token-based Splitting
# Splits text into chunks based on tokens (words or subwords), using tokenizers
# Useful for transformer models with strict token limits.
print("\n--- Using Token-based Splitting ---")
token_splitter = TokenTextSplitter(chunk_overlap=0, chunk_size=512)
token_docs = token_splitter.split_documents(documents)
create_vector_store(token_docs, "chroma_db_token")
```

9. Recursive Splitting (Splitting using Natural Boundaries such as Paragraphs, etc.)

```
# 4. Recursive Character-based Splitting
# Attempts to split text at natural boundaries (sentences, paragraphs) within character limits.
# Balances between maintaining coherence and adhering to character limits.
print("\n--- Using Recursive Character-based Splitting ---")
rec_char_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=100)
rec_char_docs = rec_char_splitter.split_documents(documents)
create_vector_store(rec_char_docs, "chroma_db_rec_char")
```

10. Similarly, we can also make our custom text splitter.

```
class CustomTextSplitter(TextSplitter):
    def split_text(self, text):
        # Custom logic for splitting text
        return text.split("\n\n") # Example: split by paragraphs

custom_splitter = CustomTextSplitter()
```

```
custom_docs = custom_splitter.split_documents(documents)
create_vector_store(custom_docs, "chroma_db_custom")
```

Embeddings

11. There are some embedding generators available on Langchain. All they do is take words and produce vectors from them. These vectors are a long series of numbers that have condensed the context of the word in the text into the embedding/vector of that word.

12. Open AI Embedding Generator

```
openai_embeddings = OpenAIEmbeddings(model="text-embedding-ada-002")
create_vector_store(docs, openai_embeddings, "chroma_db_openai")
```

13. Hugging Face Embedding Generator

```
huggingface_embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-mpnet-base-v2"
)
create_vector_store(docs, huggingface_embeddings, "chroma_db_huggingface")
```

Retreivers

14. This is how the query and the vectors (from the documents) are compared and similar data chunks are given as Outputs.

15. Getting just the top embeddings

```
# 1. Similarity Search
# This method retrieves documents based on vector similarity.
# It finds the most similar documents to the query vector based on cosine similar
# Use this when you want to retrieve the top k most similar documents.
print("\n--- Using Similarity Search ---")
```

```
query_vector_store("chroma_db_with_metadata", query,
                  embeddings, "similarity", {"k": 3})
```

16. Max Marginal Relevance. Allows you to first select a large number of similar embeddings. Then it filters them out based on the Diversity score provided by the user. A higher diversity score reduces redundancy but may also lead to unrelated embeddings.

2. Max Marginal Relevance (MMR)

```
# This method balances between selecting documents that are relevant to the query
# 'fetch_k' specifies the number of documents to initially fetch based on similarity
# 'lambda_mult' controls the diversity of the results: 1 for minimum diversity, 0 for maximum
# Use this when you want to avoid redundancy and retrieve diverse yet relevant documents
# Note: Relevance measures how closely documents match the query.
# Note: Diversity ensures that the retrieved documents are not too similar to each other,
#       providing a broader range of information.
```

```
print("\n--- Using Max Marginal Relevance (MMR) ---")
```

```
query_vector_store(
    "chroma_db_with_metadata",
    query,
    embeddings,
    "mmr",
    {"k": 3, "fetch_k": 20, "lambda_mult": 0.5},
)
```

17. Similarity Score with Threshold

3. Similarity Score Threshold

```
# This method retrieves documents that exceed a certain similarity score threshold
# 'score_threshold' sets the minimum similarity score a document must have to be retrieved
# Use this when you want to ensure that only highly relevant documents are retrieved
print("\n--- Using Similarity Score Threshold ---")
```

```
query_vector_store(
    "chroma_db_with_metadata",
    query,
```

```

embeddings,
"similarity_score_threshold",
{"k": 3, "score_threshold": 0.1},
)

```

One-off Question Using RAG

18. Get the relevant docs from the Retreiver, add them to the prompt and input it to the model to get the response.

```

# Combine the query and the relevant document contents
combined_input = (
    "Here are some documents that might help answer the question: "
    + query
    + "\n\nRelevant Documents:\n"
    + "\n\n".join([doc.page_content for doc in relevant_docs])
    + "\n\nPlease provide an answer based only on the provided documents. If the
)

# Define the messages for the model
messages = [
    SystemMessage(content="You are a helpful assistant."),
    HumanMessage(content=combined_input),
]

# Invoke the model with the combined input
result = model.invoke(messages)

```

Web Scraping

19. We could use data from the webpage to answer questions. Simply, the data is scraped from the webpage and converted to embeddings. These embeddings can then be used similar to before to answer questions.
20. Simple web-based loader

```
# Step 1: Scrape the content from apple.com using WebBaseLoader
# WebBaseLoader loads web pages and extracts their content
urls = ["https://www.apple.com/"]

# Create a loader for web content
loader = WebBaseLoader(urls)
documents = loader.load()
```

21. Fire Crawl Web Scraper

```
def create_vector_store():
    """Crawl the website, split the content, create embeddings, and persist the vector store"""
    # Define the Firecrawl API key
    api_key = os.getenv("FIRECRAWL_API_KEY")
    if not api_key:
        raise ValueError("FIRECRAWL_API_KEY environment variable not set")

    ### GIVE API_KEY , URL AND MODE TO USE FIRECRAWL
    # Step 1: Crawl the website using FireCrawlLoader
    print("Begin crawling the website...")
    loader = FireCrawlLoader(
        api_key=api_key, url="https://apple.com", mode="scrape")
    docs = loader.load()
    print("Finished crawling the website.")

    ### CHUNKING TEXT
    # Step 2: Split the crawled content into chunks
    text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=0)
    split_docs = text_splitter.split_documents(docs)

    ### DEFINING THE EMBEDDING GENERATOR
    # Step 3: Create embeddings for the document chunks
    embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

    ### INITIALIZING CHROMA WITH EMBEDDING GEN. AND VECTOR STORE (DI
```

```

### CREATING VECTOR STORE USING CHROMA
db = Chroma.from_documents(
    split_docs, embeddings, persist_directory=persistent_directory
)

### USING THE RETREIVER
# Step 5: Query the vector store
def query_vector_store(query):
    """Query the vector store with the specified question."""
    # Create a retriever for querying the vector store
    retriever = db.as_retriever(
        search_type="similarity",
        search_kwargs={"k": 3},
    )

    # Retrieve relevant documents based on the query
    relevant_docs = retriever.invoke(query)

    # Display the relevant results with metadata
    print("\n--- Relevant Documents ---")
    for i, doc in enumerate(relevant_docs, 1):
        print(f"Document {i}:\n{doc.page_content}\n")
        if doc.metadata:
            print(f"Source: {doc.metadata.get('source', 'Unknown')}\n")

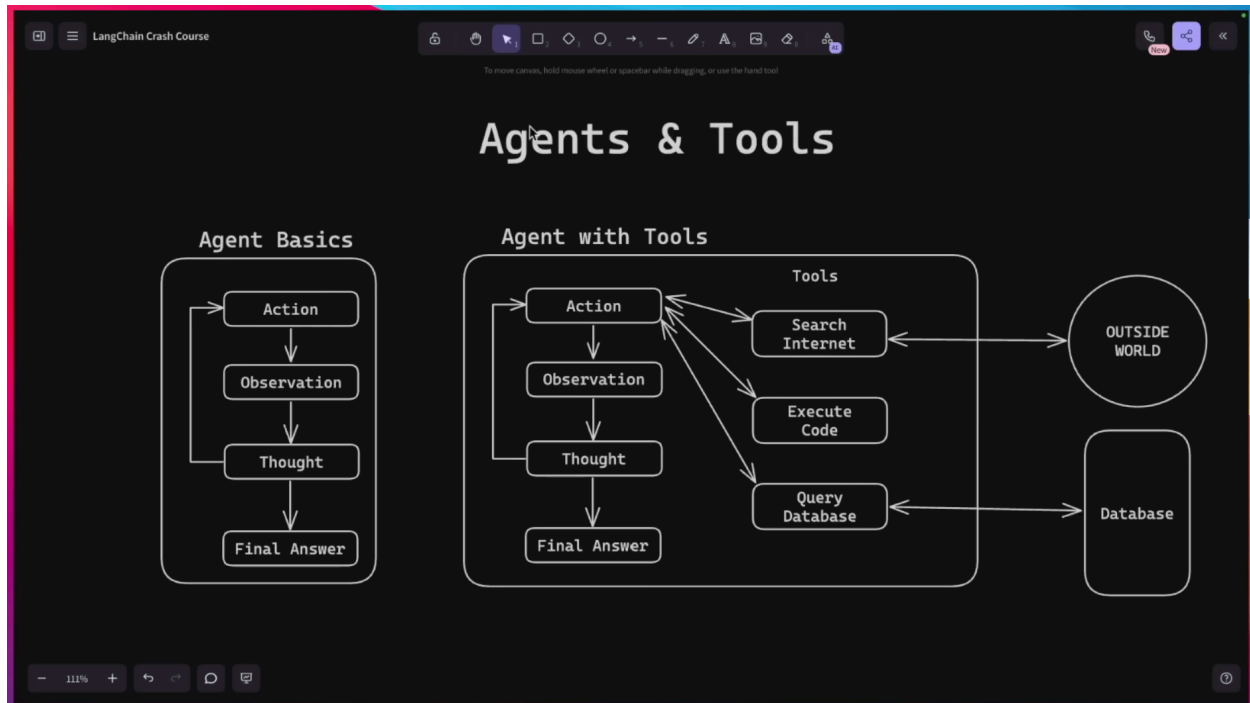
    # Define the user's question
    query = "Apple Intelligence?"

    # Query the vector store with the user's question
    query_vector_store(query)

```

Agents & Tools

1. An overview of Agentic-AI



2. Defining a Tool. The description plays a big role in letting the agent know the purpose of the tool.

```
# Define a very simple tool function that returns the current time
def get_current_time(*args, **kwargs):
    """Returns the current time in H:MM AM/PM format."""
    import datetime # Import datetime module to get current time

    now = datetime.datetime.now() # Get current time
    return now.strftime("%I:%M %p") # Format time in H:MM AM/PM format
```

```
# List of tools available to the agent
tools = [
    Tool(
        name="Time", # Name of the tool
```

```

    func=get_current_time, # Function that the tool will execute
    # Description of the tool
    description="Useful for when you need to know the current time",
),
]

```

3. Agent Creator and Executioner

```

# Create the ReAct agent using the create_react_agent function
agent = create_react_agent(
    llm=llm,
    tools=tools,
    prompt=prompt,
    stop_sequence=True,
)

# Create an agent executor from the agent and tools
agent_executor = AgentExecutor.from_agent_and_tools(
    agent=agent,
    tools=tools,
    verbose=True,
)

```

4. Prompt passed onto the agent creator:

Answer the following questions as best you can. You have access to the following

{tools}

Use the following format:

Question: the input question you must answer

Thought: you should always think about what to do

Action: the action to take, should be one of [{tool_names}]

Action Input: the input to the action

Observation: the result of the action
... (this Thought/Action/Action Input/Observation can repeat N times)
Thought: I now know the final answer
Final Answer: the final answer to the original input question

Begin!

Question: {input}
Thought:{agent_scratchpad}

5. This mainly gives a chain of thought process to the agent so that it can process information in a sequential way. {agent_scratchpad} still needs to be seen.
6. All we have to do is invoke the Agent Executioner

```
# Run the agent with a test query
response = agent_executor.invoke({"input": "What time is it?"})
```

Agents Deeper Dive

7. The true magic is due to the prompts that are passed on to the agent, I believe. Below you can see a different kind of prompt.
8. In the prompt below, some of the info acts as place holders, such as tool name and input, and the rest of it is a plain English prompt that tells the LLM to really follow a certain sequence of steps and follow them until the answer is found.

```
system
Respond to the human as helpfully and accurately as possible. You have access
{tools}
Use a json blob to specify a tool by providing an action key (tool name) and an ar
Valid "action" values: "Final Answer" or {tool_names}
Provide only ONE action per $JSON_BLOB, as shown:
...
{{
```

```

    "action": $TOOL_NAME,
    "action_input": $INPUT
  }}
  ...

```

Follow this format:

Question: input question to answer

Thought: consider previous and subsequent steps

Action:

...

\$JSON_BLOB

...

Observation: action result

... (repeat Thought/Action/Observation N times)

Thought: I know what to respond

Action:

...

```

{{
  "action": "Final Answer",
  "action_input": "Final response to human"
}}

```

Begin! Reminder to ALWAYS respond with a valid json blob of a single action. Use

9. Using RAG with agents is super simple, too. Just call RAG tools.

```

tools = [
    Tool(
        name="Answer Question",
        func=lambda input, **kwargs: rag_chain.invoke(
            {"input": input, "chat_history": kwargs.get("chat_history", [])}
        ),
        description="useful for when you need to answer questions about the conte
    )
]

```

Tools Deep Dive

10. Instead of just defining a tool and letting the agent decide what to pass into it, we can constrain the agent and let it know more details about the input arguments. Like this:

```
# StructuredTool allows us to define an input schema using Pydantic, ensuring proper
StructuredTool.from_function(
    func=concatenate_strings, # Function to execute
    name="ConcatenateStrings", # Name of the tool
    description="Concatenates two strings.", # Description of the tool
    args_schema=ConcatenateStringsArgs, # Schema defining the tool's input arguments
),

### Concatenate strings is a class and gives info regarding the input arguments
# Pydantic model for tool arguments
class ConcatenateStringsArgs(BaseModel):
    a: str = Field(description="First string")
    b: str = Field(description="Second string")

### concatenate_strings is the simple function to be called
def concatenate_strings(a: str, b: str) -> str:
    """Concatenates two strings."""
    return a + b
```

11. Using argument details is especially helpful when there are more than two input arguments.
12. You can also define the run methods of the tools, and hence define exactly what the type of inputs should be and what the type of output should be

```
class SimpleSearchTool(BaseTool):
    name = "simple_search"
    description = "useful for when you need to answer questions about current events"
    args_schema: Type[BaseModel] = SimpleSearchInput
```

```

    ### Custom RUN Method
    def _run(
        self,
        query: str,
    ) → str:
        """Use the tool."""
        from tavily import TavilyClient

        api_key = os.getenv("TAVILY_API_KEY")
        client = TavilyClient(api_key=api_key)
        results = client.search(query=query)
        return f"Search results for: {query}\n\n{results}\n"

```

```

    ### The args_schema for the tool (letting it know about the input arguments)
    class SimpleSearchInput(BaseModel):
        query: str = Field(description="should be a search query")

```

The End

Prompt Engineering

Zero Shot Prompting

1. Your Regular Prompt Without any Examples or Supporting Doc

Few-Shot Prompting

1. Giving a few examples (Input, output) and then giving it an input to get the output.

A "whatpu" is a small, furry animal native to Tanzania. An example of a sentence that uses the word whatpu is: We were traveling in Africa and we saw these

very cute whatpus. To do a "farduddle" means to jump up and down really fast. An example of a sentence that uses the word farduddle is:

Chain of Thought Prompting

1. You break down the steps of solving a problem and give it a few examples (few-shot prompting, but now with intermediate steps).

The odd numbers in this group add up to an even number: 4, 8, 9, 15, 12, 2, 1.

A: Adding all the odd numbers (9, 15, 1) gives 25. The answer is False.

The odd numbers in this group add up to an even number: 17, 10, 19, 4, 8, 12, 24.

A: Adding all the odd numbers (17, 19) gives 36. The answer is True.

The odd numbers in this group add up to an even number: 16, 11, 14, 4, 8, 13, 24.

A: Adding all the odd numbers (11, 13) gives 24. The answer is True.

The odd numbers in this group add up to an even number: 17, 9, 10, 12, 13, 4, 2.

A: Adding all the odd numbers (17, 9, 13) gives 39. The answer is False.

The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.

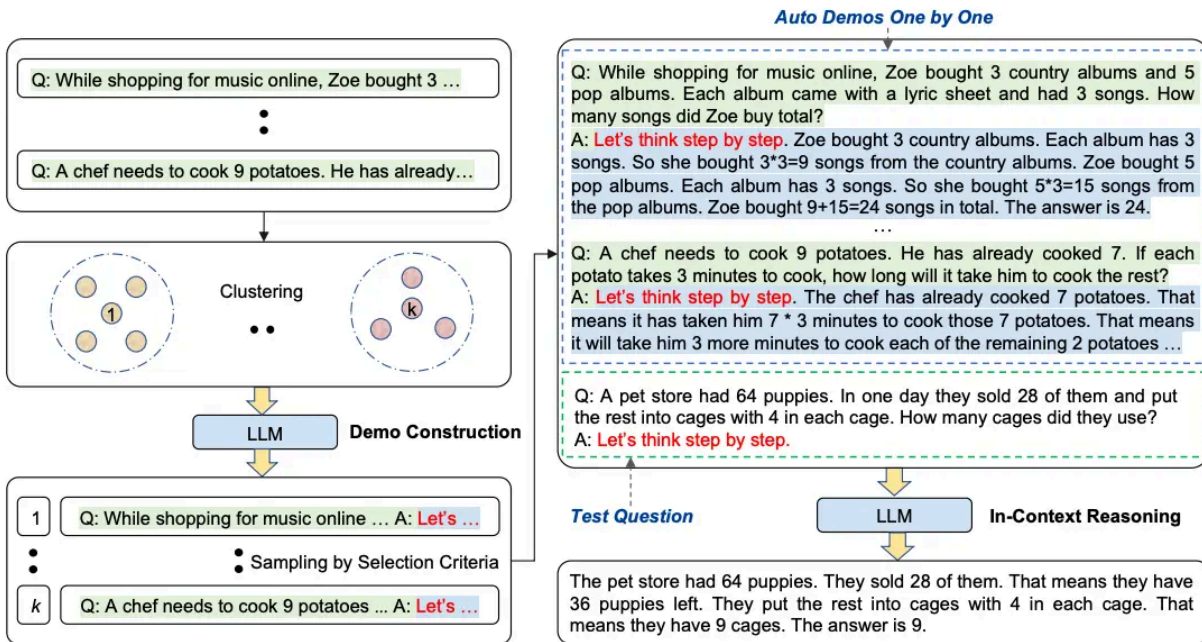
A:

2. You could also try Zero-Shot COT by just telling the model, "Let's think step by step."

AI: Let's track the apples step by step:

1. You start with 0 apples.
2. You bought 10 apples: $0 + 10 = 10$ apples.
3. You gave 2 to the neighbor: $10 - 2 = 8$ apples.
4. You gave 2 to the repairman: $8 - 2 = 6$ apples.
5. You bought 5 more apples: $6 + 5 = 11$ apples.
6. You ate 1 apple: $11 - 1 = 10$ apples.

3. We could use Automatic Chain-of-Thought (Auto-CoT) by providing intermediate steps for a few questions. Keep in mind that these questions should be diverse.



META Prompting

1. A form of prompting that focuses on the structure of the output instead of the content of the output. The structure is shown using abstract examples(unrelated to the question at hand wrt the content)
2. Focusing on the structure instead of the content allows the LLM not to overfit to the given examples.
3. META prompting does, however, assume that the model has the base knowledge of how to solve the task at hand. If the task at hand is novel the performance might deteriorate.

Self-Consistency Prompting

1. This method of prompting includes giving the model intermediate steps in various ways. Giving the model a number of examples, with each example containing a different sort of approach (intermediate steps), allows the model to choose the method of intermediate results while answering the prompt. More details are in the paper: <https://arxiv.org/abs/2311.11482>

Q: There are 15 trees in the grove. Grove workers will plant trees in the grove today. At the end of the day, there will be 21 trees. How many trees did the grove workers plant today?

A: We start with 15 trees. Later we have 21 trees. The difference must be the number of trees planted. So, they must have planted $21 - 15 = 6$ trees. The answer is 6.

Q: If there are 3 cars in the parking lot and 2 more cars arrive, how many cars are there now?

A: There are 3 cars in the parking lot already. 2 more arrive. Now there are $3 + 2 = 5$ cars. The answer is 5.

Q: Leah had 32 chocolates and her sister had 42. If they ate 35, how many pieces of chocolate does Leah still have?

A: Leah had 32 chocolates and Leah's sister had 42. That means there were originally $32 + 42 = 74$ chocolates. 35 have been eaten. So in total they still have $74 - 35 = 39$ chocolate pieces. The answer is 39.

Q: Jason had 20 lollipops. He gave Denny some lollipops. Now Jason has 12 lollipops. How many lollipops did Jason give to Denny?

A: Jason had 20 lollipops. Since he only has 12 now, he must have given the rest to Denny. The number of lollipops he has given to Denny must have been $20 - 12 = 8$ lollipops. The answer is 8.

Q: Shawn has five toys. For Christmas, he got two toys each from his mom and dad. How many toys does he have now?

A: He has 5 toys. He got 2 from mom, so after that he has $5 + 2 = 7$ toys. Then he got 2 from dad. In total he has $7 + 2 = 9$ toys. The answer is 9.

Q: There were nine computers in the server room. Five more computers were installed on Monday and Thursday. How many computers are now in the server room?

A: There are 4 days from Monday to Thursday. 5 computers were added each day. So, a total of $4 \times 5 = 20$ computers were added. There were 9 computers in the beginning, so now there are $9 + 20 = 29$ computers. The answer is 29.

Q: Michael had 58 golf balls. On tuesday, he lost 23 golf balls. On wednesday, he lost 2 more. How many golf balls did he have at the end of wednesday?

A: Michael initially had 58 balls. He lost 23 on Tuesday, so after that he has $58 - 23 = 35$ balls. On Wednesday he lost 2 more so now he has $35 - 2 = 33$ balls. The answer is 33.

Q: Olivia has \$23. She bought five bagels for \$3 each. How much money does she have left?

A: She bought 5 bagels for \$3 each. This means she spent \$15. She has \$8 left. The answer is 8.

Q: When I was 6 my sister was half my age. Now I'm 70 how old is my sister?

A: When you were 6, your sister was half your age, which is 3. The age difference between you and your sister is $6 - 3 = 3$ years. Now you are 70, so your sister is $70 - 3 = 67$ years old. The answer is 67.

General Knowledge Prompting

1. You teach the model to first produce the knowledge required to answer the question and then use that knowledge to answer. Like a two-stage process.

Getting the knowledge first allows the model to extract the knowledge regarding the subject, hence reducing the chances of wrong logical answers.

Input: Greece is larger than Mexico.

Knowledge: Greece is approximately 131,957 sq km, while Mexico is approximately 1,971,954 sq km.

Input: Glasses always fog up.

Knowledge: Condensation occurs on eyeglass lenses when water vapor from your breath hits the cool lenses.

Input: A fish is capable of thinking.

Knowledge: Fish are more intelligent than they appear. In many areas, such as memory and problem-solving, they are quite capable.

Input: A common effect of smoking lots of cigarettes in one's lifetime is a higher risk of lung cancer.

Knowledge: Those who consistently averaged less than one cigarette per day over a 50-year period had a 90% lower risk of dying from lung cancer.

Input: A rock is the same size as a pebble.

Knowledge: A pebble is a clast of rock with a particle size of 4 to 64 millimetres (1/16 to 2 1/2 inches) diameter.

Input: Part of golf is trying to get a higher point total than others.

Knowledge:

Prompt Chaining

1. You break down a long and complex prompt into sub-prompts where the output of one becomes the input to the next prompt. This helps to break down the problem and allows for easier debugging. Chain of Thought is helpful, but only if you have one complex task. If you have multiple complex tasks, it is better to break them down into sub-prompts (one per complex task).
2. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/chain-prompts>

You are a helpful assistant. Your task is to help answer a question given in a document.

####

{{document}}

####

Given a set of relevant quotes (delimited by <quotes></quotes>) extracted from a document:

####

{{document}}

####

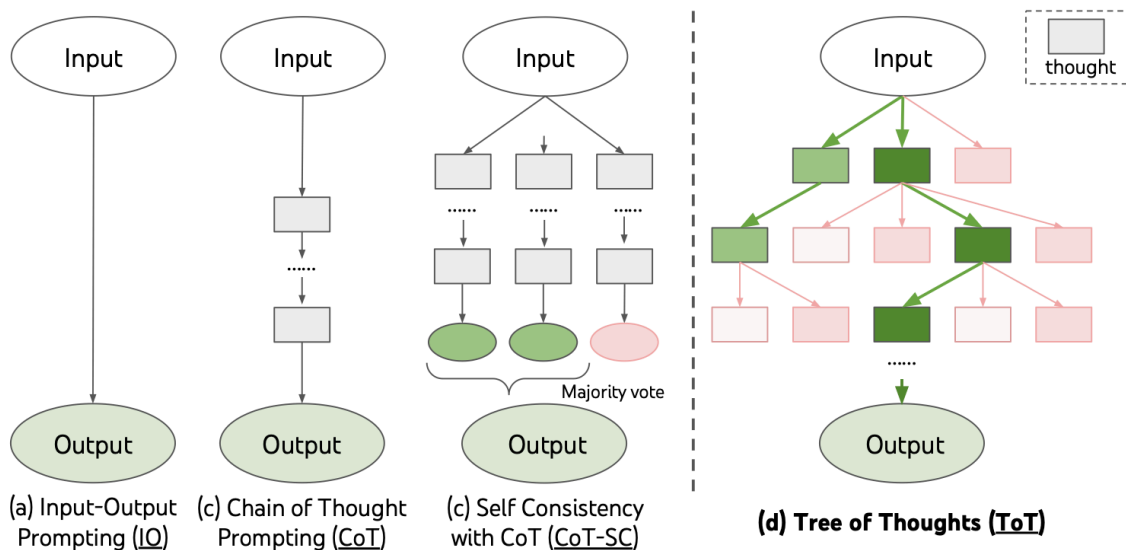
<quotes>

- Chain-of-thought (CoT) prompting[27]
- Generated knowledge prompting[37]
- Least-to-most prompting[38]
- Self-consistency decoding[39]
- Complexity-based prompting[41]
- Self-refine[42]
- Tree-of-thought prompting[43]
- Maieutic prompting[45]
- Directional-stimulus prompting[46]
- Textual inversion and embeddings[59]
- Using gradient descent to search for prompts[61][62][63][64]
- Prompt injection[65][66][67]

</quotes>

Tree of Thought Prompting

1. The model evaluates the best ROUTE from a number of possible paths. The best ROUTE can be a combination of the generated paths and hence closely mimics the human problem-solving mindset.



RAG Improvements

DATA CLEANING

1. Technically, it should work, but take care because data cleaning algorithms might remove necessary characters, which might remove meanings from words.

META-DATA

1. Adding metadata to the data chunks will help to retrieve the relevant data chunks only. The point is now, how to add metadata to the chunks of something like a research paper or a book.
2. Document loaders already add basic metadata to all chunks that contain the PDF name and similar top level stuff that can be derived via pdf name.
3. Advanced techniques include using ML data interpreters and LLMs, and OCR (Transformers) to structure the data and add additional metadata to each chunk.
4. Keywords: Nougut (META AI), Grobid

RAG HEAD TO HEAD

Docling vs PyMuPDF Loader

1. LLM 01 correctly answered 9 out of 10 questions.
2. LLM 02 correctly answered 8 out of 10 questions.
3. The critical difference lies in **Query 2 (Which Large Language Models were used)**.
 - **LLM 01 correctly identified the models used in the paper's experiments.**

- **LLM 02 incorrectly pulled models from the "Related Work" section**, which is a significant misinterpretation of the paper's methodology.
- 4. Both LLMs failed to answer Query 1 about the main research question, despite it being explicitly stated in the paper. While LLM 02 provided a more detailed answer for Query 10, LLM 01's correctness on identifying the experimental models (Query 2) is more fundamental to understanding the paper's core setup.
- 5. Therefore, due to its accuracy on a key methodological detail where LLM 02 faltered, **LLM 01 demonstrated slightly better performance**.
- 6. [Docling Wins](#)

output_01.txt

output_02.txt

Semantic vs Parent_Child vs Recursive

01 is Parent_Child, 02 is Semantic, 03 is Simple Recursive

1. **LLM 02 (output_02.txt)** demonstrated superior performance by providing the most accurate, detailed, and well-supported answers based on the content of [LLM_Prompt_Format.pdf](#). It excelled in accurately listing evaluated tasks and correctly interpreting the paper's nuanced findings, such as the non-existence of a universal prompt and the specifics of model robustness.
2. LLM 01 was a strong contender, providing accurate information on several points, but was slightly less comprehensive in listing task types and made an error in an illustrative example regarding optimal prompt formats.
3. LLM 03 was the weakest, with several inaccuracies, such as misinterpreting whether the paper identified a universal format and failing to directly answer what formats were tested.

4. Semantic Wins

output_01.txt

output_02.txt

output_03.txt

Sparse + Dense vs Dense Only

01 is Sparse + Dense

02 is Dense Only

1. In summary, **LLM 02 demonstrated superior performance** by accurately addressing all questions and often providing more detailed responses, while LLM 01 struggled to locate information for half of the queries.

2. Dense Only Wins

output_01.txt

output_02.txt

ReRank vs No ReRank

1. No ReRank Wins

-
2. Reranking takes extra time and reduces data. It might come in handy for cost efficiency, though.